

AssetPass – Blockchain-Based Digital Artwork Verification & Marketplace

Project Overview

AssetPass is a blockchain-based digital artwork management and verification system built using Solidity, Hardhat 3, TypeScript, Viem, and the Ethereum Sepolia Testnet. It provides a transparent way to register digital artwork, verify authenticity, list artwork for sale, and prevent transfer of frozen artwork.

Objectives

- Register digital artwork on the blockchain.
- Approve trusted verifiers.
- Allow approved verifiers to verify artwork.
- List verified artwork for sale.
- Allow buyers to purchase listed artwork.
- Prevent unauthorized verification.
- Prevent frozen artwork from being transferred.
- Maintain ownership and verification records on-chain.

Technologies Used

- Ethereum / Sepolia Testnet
- Solidity 0.8.34
- Hardhat 3
- Hardhat Ignition
- TypeScript
- Viem
- Node.js and npm
- MetaMask
- Alchemy
- Remix IDE

Project Structure

```
AssetPass/  
├── contracts/AssetPass.sol  
├── ignition/modules/AssetPass.ts  
├── test/AssetPass.ts  
├── scripts/  
└── artifacts/
```

```
|— cache/
|— node_modules/
|— .env
|— .gitignore
|— hardhat.config.ts
|— package.json
|— package-lock.json
|— tsconfig.json
|— README.md
```

Smart Contract

The main smart contract is `contracts/AssetPass.sol`. It manages artwork registration, verification, listing, purchasing, ownership and transfer restrictions.

Roles

- Owner – manages the platform and approves verifiers.
- Verifier – an approved account that verifies artwork.
- Artist – registers artwork.
- Buyer – purchases verified artwork.
- Smart Contract – controls the workflow and stores relevant blockchain records.

How AssetPass Works

Owner → Approve Verifier → Artist Registers Artwork → Approved Verifier Verifies Artwork → Artwork Becomes Eligible for Listing → Artist Lists Artwork → Buyer Purchases Artwork → Ownership Transfer

If artwork is frozen, the smart contract prevents transfer according to the conditions implemented in the contract.

Main Features

1. Set Verifier

The owner can approve a verifier. Function: `setVerifier(address verifier, bool approved)`.

2. Register Artwork

An artist can register artwork. The artwork receives a unique token ID and relevant information is stored in the contract.

3. Verify Artwork

An approved verifier can verify registered artwork. Unauthorized accounts cannot perform this operation.

4. Approve

The owner can approve another address for a particular token according to the ERC-721 mechanism. Function: approve(address to, uint256 tokenId).

5. List Artwork

Verified artwork can be listed for sale.

6. Purchase Artwork

A buyer can purchase listed artwork and ownership is transferred through the smart contract.

7. Frozen Artwork Protection

Frozen artwork cannot be transferred when the contract's freeze condition is active.

Installation

Prerequisites

- Node.js
- npm
- VS Code
- MetaMask

Install Dependencies

npm install

Build / Compile

npx hardhat build

Environment Configuration

Create a .env file:

```
SEPOLIA_RPC_URL=YOUR_SEPOLIA_RPC_URL  
SEPOLIA_PRIVATE_KEY=YOUR_PRIVATE_KEY
```

Never commit the real private key to GitHub. Keep .env in .gitignore.

Sepolia RPC URL

Use an Alchemy Sepolia endpoint in the form:

`https://eth-sepolia.g.alchemy.com/v2/YOUR_ALCHEMY_API_KEY`

Deployment

The project uses Hardhat Ignition. Deployment module: `ignition/modules/AssetPass.ts`

Local Deployment

`npx hardhat ignition deploy ignition/modules/AssetPass.ts`

Sepolia Deployment

`npx hardhat ignition deploy ignition/modules/AssetPass.ts --network sepolia`

After successful deployment, Hardhat displays the deployed AssetPass contract address.

Testing

Run all tests with:

`npx hardhat test`

The project tests deployment, verifier approval, artwork registration, verification, unauthorized verification prevention, listing, purchasing, and frozen artwork protection.

Test result: 8 passing (8 nodes).

Contract Interaction

- Compile AssetPass.sol using Solidity 0.8.34.
- Open Deploy & Run Transactions.
- Select Browser Extension and connect MetaMask.
- Make sure MetaMask is connected to Sepolia.
- Open the deployed AssetPass contract.
- Interact with the available functions.

Important Functions

- `setVerifier`
- `registerArtwork`
- `verifyArtwork`
- `approve`
- `listArtwork`

Demonstration Flow

- Owner approves a verifier using setVerifier.
- Artist registers artwork.
- Approved verifier verifies artwork.
- Token approval is performed where required.
- Verified artwork is listed for sale.
- Buyer purchases the listed artwork.
- The smart contract transfers ownership and enforces its restrictions.

Security Considerations

- Owner-controlled verifier approval.
- Restricted verification access.
- Blockchain-based ownership records.
- Verification status stored on-chain.
- Frozen artwork protection.
- Smart contract-controlled transfers.
- Private key stored outside source code.

Security Audit

This is a college/academic demonstration project. No professional third-party security audit has been performed on the AssetPass smart contract. Automated tests validate implemented functionality but are not equivalent to a professional security audit. A professional audit would be recommended before production deployment.

Testing Summary

Test	Result
Deploy AssetPass contract	Passed
Owner adds approved verifier	Passed
Artist registers artwork	Passed
Approved verifier verifies artwork	Passed
Unauthorized user cannot verify artwork	Passed
Verified artwork can be listed	Passed
Buyer can purchase artwork	Passed
Frozen artwork cannot be sold	Passed

Future Improvements

- Professional smart contract security audit.
- Frontend web application.
- IPFS integration.
- NFT metadata management.

- Royalty distribution.
- Marketplace dashboard.
- Advanced role-based access control.
- Artwork dispute management.
- Event-based transaction history.
- Multi-signature administration.
- Production-grade monitoring.
- Mainnet deployment after security auditing.

Academic Purpose

AssetPass was developed as an academic blockchain project to demonstrate smart contract development, Solidity programming, Ethereum blockchain interaction, ERC-721 based digital asset management, access control, verification workflows, blockchain transactions, Hardhat development, automated testing, and Sepolia testnet deployment.

Commands Summary

```
npm install
```

```
npx hardhat build
```

```
npx hardhat test
```

```
npx hardhat ignition deploy ignition/modules/AssetPass.ts
```

```
npx hardhat ignition deploy ignition/modules/AssetPass.ts --network sepolia
```

Important Files

- contracts/AssetPass.sol – main smart contract.
- ignition/modules/AssetPass.ts – Hardhat Ignition deployment module.
- test/AssetPass.ts – automated tests.
- hardhat.config.ts – compiler and network configuration.
- .env – sensitive environment/configuration variables.

Disclaimer

AssetPass is an educational prototype developed for academic purposes. It is deployed on the Ethereum Sepolia test network and is not a production-ready commercial marketplace. No real-world artwork ownership, legal ownership, or financial value should be inferred from the testnet implementation.

Project Information

- Project Name: AssetPass
- Type: Blockchain / Smart Contract / Digital Artwork Verification & Marketplace
- Blockchain: Ethereum Sepolia Testnet
- Smart Contract Language: Solidity
- Development Framework: Hardhat 3
- Testing: Node.js native test runner
- Blockchain Library: Viem
- Wallet: MetaMask
- RPC Provider: Alchemy
- Purpose: Academic / College Demonstration